



**Institute of Technology of Cambodia**  
**International Program**

# **Parking Management System**

Project Report

Data Structure and Algorithms

Lecturer: Seak Leng

Submission Date: June 28, 2026

| Name               | Student ID | Role                              |
|--------------------|------------|-----------------------------------|
| Chheng Sokuntheary | p20240044  | Linked List, Sorting, Integration |
| Chin Menghong      | p20240024  | Stack, Queue, File I/O            |
| Dara Panhaseth     | p22040002  | Hash Table, Binary Search Tree    |

## 1. Introduction

This report documents the design, implementation, and testing of a Parking Management System built in C++ for the Data Structure and Algorithms course. The system simulates the day-to-day operation of a university parking lot, where vehicles arrive, occupy a slot, and eventually leave, and where the lot itself has a limited number of spaces for each vehicle type.

structures implemented from scratch: a singly linked list, a stack, a queue, a hash table, a binary search tree, and a bubble sort algorithm applied directly on the linked list. Each structure was chosen deliberately to match a specific real-world need within the system, rather than being added only to satisfy a checklist of requirements.

The project was completed by a team of three members over a two-week period, with each member taking ownership of a distinct part of the system and integrating their work through a shared menu-driven interface in `main.cpp`.

## 2. Problem Statement

A university parking lot with a fixed number of slots faces several recurring operational problems that a simple paper log cannot solve efficiently:

- Finding whether a specific vehicle is currently parked, and where, must be fast even as the number of records grows.
- When the lot is full, vehicles need a fair, predictable way to wait for the next available slot rather than being turned away outright.
- Mistakes happen: a vehicle parked at the wrong slot, or removed in error — and the system should allow the most recent action to be undone without manual cleanup.
- Staff reviewing the lot may want to see vehicles in a specific order, whether by slot number, plate, or arrival time.
- All of this information must persist between sessions, since the program will be closed and reopened many times throughout a working day.

The system addresses each of these problems directly: a hash table for fast lookup, a queue for fair waiting, a stack for undo, a binary search tree for ordered browsing, sorting for flexible ordering, and file input and output for persistence.

## 3. System Design

### 3.1 Overview

The system is a console-based, menu-driven application. On startup, it loads any previously saved data from `parking_data.csv` into three live structures: the linked list, the hash table, and the binary search tree. The user is then presented with a numbered menu of sixteen operations, and the program loops on this menu until the user chooses to save and exit.

### 3.2 Slot Allocation

The lot is divided into 300 total slots. Slots 1 through 10 are reserved for cars, and slots 11 through 300 are reserved for motorcycles. This split reflects a deliberate design decision favoring motorcycle capacity, which is common on university campuses where motorcycles make up the majority of student vehicles.

### 3.3 Module Breakdown

| File              | Responsibility                                 | Owner    |
|-------------------|------------------------------------------------|----------|
| vehicle.h         | Shared Vehicle struct used by all modules      | Shared   |
| linkedList.h/.cpp | Stores all parked vehicle records              | Member A |
| sorting.h/.cpp    | Bubble sort by plate, slot, or time            | Member A |
| stack.h/.cpp      | Undo functionality and action history          | Member B |
| queue.h/.cpp      | Waiting list for vehicles when the lot is full | Member B |
| fileio.h/.cpp     | Save and load all vehicle records as CSV       | Member B |

|                  |                                                  |          |
|------------------|--------------------------------------------------|----------|
| hashtable.h/.cpp | Fast search, update, and removal by plate number | Member C |
| bst.h/.cpp       | Ordered browsing of occupied slots               | Member C |
| main.cpp         | Menu and integration of all modules              | Member A |

### 3.4 System Flow

Every park or remove action updates three structures at once: the linked list, the hash table, and the binary search tree, so that all three always remain in agreement with each other. The stack records the action afterward so it can be reversed by the undo feature. When a vehicle cannot be parked because its slot range is full, it is placed into the queue instead of being rejected outright, and the system automatically promotes the next matching vehicle in the queue whenever a slot of the matching type becomes free.

## 4. Data Structures Used

| Data Structure        | Purpose in the System                                                                       | Implemented By |
|-----------------------|---------------------------------------------------------------------------------------------|----------------|
| Singly Linked List    | Holds every parked vehicle as a node, supporting add, delete, search, and update by plate   | Member A       |
| Sorting (Bubble Sort) | Orders the linked list by plate, slot ID, or entry time on demand                           | Member A       |
| Stack                 | Records the most recent park or remove action so it can be undone, and shows action history | Member B       |
| Queue                 | Holds vehicles waiting for a free slot in first-in, first-out order                         | Member B       |

|                    |                                                                                            |          |
|--------------------|--------------------------------------------------------------------------------------------|----------|
| Hash Table         | Provides near-instant lookup, update, and deletion by plate number using separate chaining | Member C |
| Binary Search Tree | Keeps every parked vehicle organized by slot ID so the lot can be browsed in sorted order  | Member C |

Each structure was selected because it is the most natural fit for one specific operation in the system, rather than being forced to satisfy the assignment's requirement list. A parking lot lane behaves like a stack when only the most recent action needs to be reversed; a waiting line behaves like a queue; and a directory of vehicles searched by plate behaves like a hash table.

## 5. Algorithms Used

### 5.1 Hash Function

The hash function sums the ASCII value of every character in a vehicle's plate string, then takes the result modulo the table size, to produce a bucket index. Separate chaining is used to resolve collisions, meaning each bucket is itself the head of a small linked list of vehicles that hashed to the same index.

### 5.2 Bubble Sort

Sorting is implemented as a direct, in-place bubble sort on the linked list, swapping the data stored in adjacent nodes rather than rearranging pointers. Three variants exist, differing only in the comparison used: by plate string, by slot ID, and by entry time string. The algorithm includes an early-exit flag, so a pass with no swaps stops the sort immediately rather than running every remaining pass unnecessarily.

### 5.3 Binary Search Tree Operations

Insertion, search, and deletion on the tree all follow the standard recursive binary search tree pattern: a vehicle with a smaller slot ID than the current node goes to the left subtree, and a larger one goes to the right. Deletion handles all three classic cases — a node with no children is removed directly, a node with one child is replaced by that child, and a node with two children has its data replaced by the minimum value from its right subtree before that minimum node is removed.

## 5.4 Tree Traversals

Three traversal orders are implemented. In-order traversal, which visits the left subtree, then the current node, then the right subtree, is used to browse all occupied slots from smallest to largest with no separate sorting step required. Pre-order and post-order traversals are also implemented to demonstrate a complete understanding of all three traversal types taught in the course.

## 6. Complexity Analysis

| Operation                           | Average Case           | Worst Case             |
|-------------------------------------|------------------------|------------------------|
| Linked list insert at end           | $O(1)$                 | $O(1)$                 |
| Linked list search or delete        | $O(n)$                 | $O(n)$                 |
| Bubble sort                         | $O(n \text{ squared})$ | $O(n \text{ squared})$ |
| Stack push / pop / peek             | $O(1)$                 | $O(1)$                 |
| Queue enqueue / dequeue             | $O(1)$                 | $O(1)$                 |
| Hash table insert / search / delete | $O(1)$                 | $O(n)$                 |
| BST insert / search / delete        | $O(\log n)$            | $O(n)$                 |
| BST traversal (in, pre, post order) | $O(n)$                 | $O(n)$                 |

The hash table's worst case of  $O(n)$  would occur only if every plate number happened to hash to the same bucket, which is unlikely in practice given the variety of characters in

real plate numbers. The binary search tree's worst case of  $O(n)$  would occur if vehicles were parked in strictly increasing or decreasing slot order, causing the tree to degrade into a structure resembling a linked list. This tradeoff is discussed further in the Conclusion.

## 7. Testing Results

### 7.1 Test Cases

| Test Case                                            | Expected Result                                                | Actual Result |
|------------------------------------------------------|----------------------------------------------------------------|---------------|
| Park a new vehicle with a free slot available        | Vehicle assigned the next free slot of the correct type        | Pass          |
| Park a vehicle with a plate already parked           | System rejects the duplicate and shows the existing slot       | Pass          |
| Park a vehicle when all matching slots are full      | Vehicle is placed into the waiting queue                       | Pass          |
| Remove a parked vehicle while the queue is non-empty | Slot freed, and the next matching vehicle in queue is promoted | Pass          |
| Undo the most recent park action                     | Vehicle removed from list, hash table, and BST                 | Pass          |
| Undo the most recent remove action                   | Vehicle restored to its original slot in all three structures  | Pass          |
| Search for a plate that exists                       | Full vehicle details returned instantly                        | Pass          |
| Search for a plate that does not exist               | Clear not-found message, no crash                              | Pass          |

|                                           |                                                           |      |
|-------------------------------------------|-----------------------------------------------------------|------|
| Sort by plate, slot ID, and entry time    | List re-orders correctly for each option                  | Pass |
| Browse slots using in-order BST traversal | Slots listed in ascending order with no extra sorting     | Pass |
| Save and exit, then reopen the program    | All previously parked vehicles reload correctly from CSV  | Pass |
| Update a vehicle's entry time             | New time reflected in both the linked list and hash table | Pass |

## 7.2 Known Limitations

Testing surfaced a small number of edge cases that the team chose to document rather than resolve, given the project timeline:

- When a freed slot does not match the vehicle type at the front of the waiting queue, that vehicle is re-added to the back of the queue rather than retaining its original position. In a busy lot with several waiting vehicles of mixed types, this could allow a later arrival to be served before an earlier one.
- The waiting queue itself is not saved to the CSV file. Only parked vehicles persist between sessions; any vehicle still waiting in the queue at the time the program is closed is lost on the next startup.
- Plate matching throughout the system is case-sensitive, so a plate entered in lowercase will not match the same plate stored in uppercase.
- The binary search tree is not self-balancing, so its worst-case performance depends on the order in which vehicles are parked.

## 8. Conclusion

The Parking Management System successfully demonstrates all six required data structures working together within a single coherent application, rather than as six isolated demonstrations. Every structure was assigned a role that matches a genuine need in the



problem domain: a hash table where speed of lookup matters most, a queue where fairness of order matters most, a stack where reversibility matters most, and a binary search tree where ordered traversal matters most.

The three-member team divided the work along clean module boundaries, with each member owning complete, independently testable files that were integrated through a single shared menu in main.cpp. This division allowed each member to test and demonstrate their own contribution clearly while still producing one unified, working system.

Given more time, the system could be extended in several directions: replacing the plain binary search tree with a self-balancing variant such as an AVL tree to guarantee logarithmic performance in all cases, extending the CSV format to persist the waiting queue as well as parked vehicles, and adding case-insensitive plate matching. These improvements were considered but intentionally left out of scope to keep the implementation aligned with the structures taught in the course and to allow the team to focus its remaining time on thorough testing rather than additional features.

---

Youtube Video: <https://youtu.be/9L8CN2crk98>

Git: [https://github.com/sokuntheary-chheng/DSA\\_Project\\_Parking\\_Management.git](https://github.com/sokuntheary-chheng/DSA_Project_Parking_Management.git)